

-Kabir Sinha

-E23CSEU1343

CSET419 – Introduction to Generative AI

Lab 10: Sequential Data Generation using LSTM and Transformer with Positional Encoding

```
[1]  import torch
    import torch.nn as nn
    import math

[2] text = """
    artificial intelligence systems learn patterns from data.
    sequence models process information step by step.
    recurrent neural networks are useful for sequence prediction.
    lstm networks handle long term dependencies.

    deep learning models improve sequence learning.
    generative models create new samples from learned patterns.
    language models predict the next word in a sentence.
    sequence generation is used in chatbots and assistants.

    machine learning helps computers learn automatically.
    training data improves model accuracy.
    neural networks simulate human brain structures.
    optimization algorithms improve learning efficiency.

    technology is transforming modern education.
    online learning platforms use artificial intelligence.
    students benefit from intelligent tutoring systems.
    automation improves productivity and decision making.
    """
```

[3]
✓ Os

```
words = text.lower().split()
vocab = sorted(set(words))

word2idx = {w:i for i,w in enumerate(vocab)}
idx2word = {i:w for w,i in word2idx.items()}

encoded = [word2idx[w] for w in words]

seq_length = 5
X, y = [], []

for i in range(len(encoded)-seq_length):
    X.append(encoded[i:i+seq_length])
    y.append(encoded[i+seq_length])

X = torch.tensor(X)
y = torch.tensor(y)
```

[4]
✓ Os

```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embed_size=64, hidden_size=128):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out

lstm_model = LSTMModel(len(vocab))
```

[5]

✓ 8s

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(lstm_model.parameters(), lr=0.01)

for epoch in range(200):
    outputs = lstm_model(X)
    loss = criterion(outputs, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 50 == 0:
        print(f"LSTM Epoch {epoch}, Loss: {loss.item():.4f}")
```

▼

```
LSTM Epoch 0, Loss: 4.4150
LSTM Epoch 50, Loss: 0.0004
LSTM Epoch 100, Loss: 0.0002
LSTM Epoch 150, Loss: 0.0002
```

[6]

✓ 0s

```
def generate_text(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()

    for _ in range(length):
        x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])
        pred = model(x)
        next_word = idx2word[pred.argmax().item()]
        words_seed.append(next_word)

    return " ".join(words_seed)

print(generate_text(lstm_model, "artificial intelligence systems"))
```

▼

```
artificial intelligence systems from data. from data. sequence models process information step by
```

[7]
✓ 0s

```
class PositionalEncoding(nn.Module):
    def __init__(self, embed_size, max_len=100):
        super().__init__()
        pe = torch.zeros(max_len, embed_size)

        for pos in range(max_len):
            for i in range(0, embed_size, 2):
                pe[pos, i] = math.sin(pos / (10000 ** (i / embed_size)))
                if i+1 < embed_size:
                    pe[pos, i+1] = math.cos(pos / (10000 ** (i / embed_size)))

        self.pe = pe.unsqueeze(0)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

[8]
✓ 0s

```
class TransformerModel(nn.Module):
    def __init__(self, vocab_size, embed_size=64, heads=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.pos_encoding = PositionalEncoding(embed_size)
        self.attention = nn.MultiheadAttention(embed_size, heads, batch_first=True)
        self.fc = nn.Linear(embed_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoding(x)
        attn_output, _ = self.attention(x, x, x)
        out = self.fc(attn_output[:, -1, :])
        return out

transformer = TransformerModel(len(vocab))
```

[9]
✓ 1s

```
optimizer = torch.optim.Adam(transformer.parameters(), lr=0.01)

for epoch in range(200):
    outputs = transformer(X)
    loss = criterion(outputs, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 50 == 0:
        print(f"Transformer Epoch {epoch}, Loss: {loss.item():.4f}")
```

▼

```
Transformer Epoch 0, Loss: 4.4752
Transformer Epoch 50, Loss: 0.0002
Transformer Epoch 100, Loss: 0.0001
Transformer Epoch 150, Loss: 0.0000
```

```
[10]
✓ Os ▶ def generate_text_tf(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()

    for _ in range(length):
        x = torch.tensor([word2idx[w] for w in words_seed[-seq_length:]])
        pred = model(x)
        next_word = idx2word[pred.argmax().item()]
        words_seed.append(next_word)

    return " ".join(words_seed)

print(generate_text_tf(transformer, "sequence models process"))

... sequence models process step. recurrent in a recurrent generation models sequence the step.
```

import torch

import torch.nn as nn

import math

text = ""

artificial intelligence systems learn patterns from data.

sequence models process information step by step.

recurrent neural networks are useful for sequence prediction.

lstm networks handle long term dependencies.

deep learning models improve sequence learning.

generative models create new samples from learned patterns.

language models predict the next word in a sentence.

sequence generation is used in chatbots and assistants.

machine learning helps computers learn automatically.

training data improves model accuracy.

neural networks simulate human brain structures.

optimization algorithms improve learning efficiency.

technology is transforming modern education.

online learning platforms use artificial intelligence.

students benefit from intelligent tutoring systems.

automation improves productivity and decision making.

"""

```
words = text.lower().split()
```

```
vocab = sorted(set(words))
```

```
word2idx = {w:i for i,w in enumerate(vocab)}
```

```
idx2word = {i:w for w,i in word2idx.items()}
```

```
encoded = [word2idx[w] for w in words]
```

```
seq_length = 5
```

```
X, y = [], []
```

```
for i in range(len(encoded)-seq_length):
```

```
    X.append(encoded[i:i+seq_length])
```

```
    y.append(encoded[i+seq_length])
```

```
X = torch.tensor(X)
```

```
y = torch.tensor(y)
```

```
class LSTMModel(nn.Module):
```

```
    def __init__(self, vocab_size, embed_size=64, hidden_size=128):
```

```
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embed_size)
```

```
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
```

```
        self.fc = nn.Linear(hidden_size, vocab_size)
```

```
    def forward(self, x):
```

```
        x = self.embedding(x)
```

```
        out, _ = self.lstm(x)
```

```
        out = self.fc(out[:, -1, :])
```

```
        return out
```

```
lstm_model = LSTMModel(len(vocab))
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(lstm_model.parameters(), lr=0.01)
```

```
for epoch in range(200):
```

```
    outputs = lstm_model(X)
```

```
loss = criterion(outputs, y)
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

```
if epoch % 50 == 0:
```

```
    print(f"LSTM Epoch {epoch}, Loss: {loss.item():.4f}")
```

```
def generate_text(model, seed, length=10):
```

```
    model.eval()
```

```
    words_seed = seed.lower().split()
```

```
    for _ in range(length):
```

```
        x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]])
```

```
        pred = model(x)
```

```
        next_word = idx2word[pred.argmax().item()]
```

```
        words_seed.append(next_word)
```

```
    return " ".join(words_seed)
```

```
print(generate_text(lstm_model, "artificial intelligence systems"))
```



```

class PositionalEncoding(nn.Module):

    def __init__(self, embed_size, max_len=100):
        super().__init__()

        pe = torch.zeros(max_len, embed_size)

        for pos in range(max_len):
            for i in range(0, embed_size, 2):
                pe[pos, i] = math.sin(pos / (10000 ** (i / embed_size)))
                if i+1 < embed_size:
                    pe[pos, i+1] = math.cos(pos / (10000 ** (i / embed_size)))

        self.pe = pe.unsqueeze(0)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

```

```

class TransformerModel(nn.Module):

    def __init__(self, vocab_size, embed_size=64, heads=2):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.pos_encoding = PositionalEncoding(embed_size)
        self.attention = nn.MultiheadAttention(embed_size, heads, batch_first=True)
        self.fc = nn.Linear(embed_size, vocab_size)

    def forward(self, x):

```

```
x = self.embedding(x)
x = self.pos_encoding(x)
attn_output, _ = self.attention(x, x, x)
out = self.fc(attn_output[:, -1, :])
return out
```

```
transformer = TransformerModel(len(vocab))
```

```
optimizer = torch.optim.Adam(transformer.parameters(), lr=0.01)
```

```
for epoch in range(200):
    outputs = transformer(X)
    loss = criterion(outputs, y)
```

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

```
if epoch % 50 == 0:
    print(f"Transformer Epoch {epoch}, Loss: {loss.item():.4f}")
```

```
def generate_text_tf(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()
```

```
for _ in range(length):  
    x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])  
    pred = model(x)  
    next_word = idx2word[pred.argmax().item()]  
    words_seed.append(next_word)  
  
return " ".join(words_seed)
```

```
print(generate_text_tf(transformer, "sequence models process"))
```